PROJECT BLUEPRINT · JUNE 2026

JobScout

An automated, CV-driven job recommender

From a fixed list of companies to a ranked shortlist of roles that fit you — refreshed automatically, every day.

A blueprint for building the tool end to end.

Contents

01 The vision
02 The problem it solves
03 The solution at a glance
04 How it works: the pipeline
05 The core insight: tap feeds, don’t scrape
06 The matching engine
07 The application: your dashboard
08 Triggers & automation
09 The first proof of concept
10 What it costs
11 Risks & mitigations
12 Roadmap
A Appendix A — ATS connector reference
B Appendix B — suggested tech stack

01 The vision

Job hunting is a research problem disguised as a waiting game. You already know the companies you would love to work for — perhaps a few hundred of them. The roles that fit you appear on their career pages at unpredictable times, sit there for a few weeks, and quietly disappear. Finding them means checking the same sites over and over, reading mostly irrelevant postings, and trying to remember what you have already seen.

JobScout removes that grind. You upload your CV once, name the companies and locations you care about, and from then on a personal agent watches those companies for you. Every day it pulls their newest openings, reads each one against your profile, and surfaces only the roles worth your attention — ranked, dated, and explained. You open a clean dashboard, see what is new, and mark each role as applied, skipped, or rejected. Nothing else competes for your attention.

> **The core idea:** you decide **where** you want to work. JobScout handles the **watching, filtering, and tracking** — the parts that are pure, repetitive effort.

Why this is the right moment

Three things make this practical in 2026 that were not a few years ago. Language models can now judge the fit between a CV and a job description cheaply and well. Most companies publish their openings through a handful of recruiting platforms that expose clean, structured data. And lightweight automation lets the whole loop run on a schedule, or on demand from your phone — with no servers to babysit.

At a glance	
What it does	Watches your target companies and recommends only the roles that fit your CV and goals.
Why it is reliable	Reads structured job feeds from recruiting platforms — not fragile page scrapes.
Why it is cheap	Feed pulls are free; AI runs only on genuinely new postings — cents per day.
How it runs	A daily scheduled job, or triggered from your phone. Hands-off.

02 The problem it solves

A focused job search is surprisingly hostile to doing by hand:

- **Fragmentation.** Your targets live on hundreds of separate career sites, each with its own layout and no shared feed you can subscribe to.

- **Timing.** Good roles are posted and filled on their own schedule. Miss the window and the posting is gone.
- **Noise.** Most openings on any given company page are irrelevant to you. Skimming them is slow and demoralising.
- **No memory.** There is no single place that remembers what you have seen, applied to, or deliberately passed on.

The result is that strong candidates apply to fewer, later, and worse-matched roles than they could — not for lack of ability, but for lack of a system. The bottleneck is never deciding where to work; it is the monitoring, filtering, and tracking in between. That is exactly the part a machine should own.

03 The solution at a glance

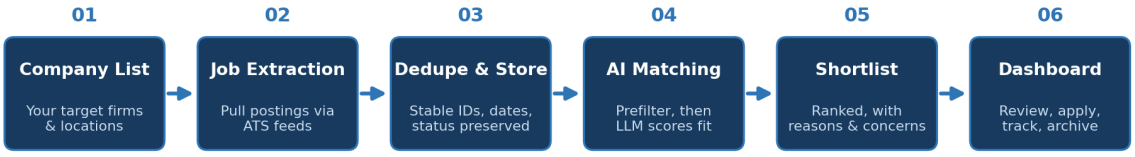
JobScout is a personal, always-on job scout built around three pillars:

1. **Reliable monitoring.** It keeps a live picture of every opening at your target companies, pulled from structured recruiting feeds so the data is clean and consistently dated.
2. **Intelligent matching.** It reads each new opening against your CV and your own notes about what you want, scoring fit and explaining its reasoning — so you always see why a role surfaced.
3. **Effortless tracking.** A single dashboard shows what is new, lets you act in one click, and quietly remembers everything you have applied to, skipped, or rejected.

> **In one line:** you supply a CV and a company list; JobScout supplies a ranked, up-to-date shortlist of roles that fit — and a place to manage them.

04 How it works: the pipeline

The system is a simple, one-directional pipeline. Each stage does one job and hands clean data to the next.



Runs automatically on a schedule — or on demand from your phone

Company list. The single input you maintain: the companies you would consider, plus the locations that matter. Yours to grow or prune at any time.

Job extraction. For each company, JobScout fetches the current openings directly from the recruiting platform that company uses. This is the make-or-break stage — the next section explains how it stays reliable across hundreds of firms.

Dedupe & store. Every posting carries a stable identifier from its source. JobScout uses it to recognise what it has already seen, so each role is processed once, your decisions persist as the feed refreshes, and the original posting date is preserved.

AI matching. New postings pass through a two-stage filter: a cheap first pass discards the obviously irrelevant, then a language model scores the rest against your CV and notes, with a short rationale.

Shortlist. The surviving roles become dated, ranked recommendations, each with a fit score and a sentence on why it fits and what to watch for.

Dashboard. The shortlist lands in your dashboard, where you review, apply, and track — closing the loop.

05 The core insight: tap feeds, don't scrape

The hardest part of this project is not the AI — it is getting clean, reliably dated job data out of hundreds of different company websites. The obvious approach, pointing a scraper or a browser agent at 500 career pages, is also the worst: every site is laid out differently, layouts change without warning, posting dates are often missing from the page, and running a browser against 500 sites on a schedule is slow and expensive.

There is a much better path. Almost no company builds its career page by hand. The overwhelming majority run their hiring on a handful of Applicant Tracking Systems (ATS) — platforms like Greenhouse, Lever, Personio, Workday, SmartRecruiters, and Ashby. Crucially, each of these exposes a structured, machine-readable feed of a company's open roles, usually as JSON and usually without authentication. These feeds hand you exactly the fields you need — title, location, posting date, description, and a stable job ID — in a consistent shape.

> **The decision that makes or breaks it:** instead of 500 fragile scrapers, you write about **eight resilient connectors** — one per ATS — and let them cover the large majority of your companies.

The pipeline therefore starts with a one-time classification step: for each company, detect which ATS it uses (its career URL usually gives it away) and store that. From then on, fetching a company's jobs is a single, fast, structured request. Realistically, eight connectors cover roughly 70–85% of a typical target list. The remaining long tail — companies on bespoke or obscure systems — is handled by a fallback cascade, described below, that turns to custom code only as a last resort.

This one decision is what makes the whole project both reliable and cheap. It solves the “dates in a reliable format” problem outright, gives you the stable IDs your status tracking depends on, and replaces hundreds of brittle scrapers with a small, maintainable set of connectors.

The major feeds, and what they cover

Platform	Public endpoint (shape)	Auth	Notes
Greenhouse	`boards-api.greenhouse.io/v1/boards/{co}/jobs?content=true`	None	JSON; full descriptions with content=true
Lever	`api.lever.co/v0/postings/{co}?mode=json`	None	JSON; filter by team, location, level
Personio	`{co}.jobs.personio.de/xml?language=en`	None	XML; dominant in DACH (some use .com)
Ashby	`api.ashbyhq.com/posting-api/job-board/{co}`	None	JSON; one call returns the whole board
SmartRecruiters	`api.smartrecruiters.com/v1/companies/{co}/postings`	None*	Published roles; filter by city, country, dept
Workday	`{tenant}.wdN.myworkdayjobs.com/.../{site}/jobs`	None	POST with JSON body; data centre varies
+ others	`Recruitee, Workable, Teamtailor, Join, BambooHR …`	varies	Add connectors as your list demands

*Endpoint formats verified against each platform’s official developer documentation, June 2026. *SmartRecruiters returns published postings without a key. Always confirm a company’s live career URL before hardcoding its identifier.*

Handling the long tail: a fallback cascade

The connectors above cover most of your list — but not all. Rather than writing a custom scraper for every exception, the pipeline tries a cascade of extractors, cheapest and most reliable first, and reaches for bespoke code only as a last resort. The key realisation: most “no-

ATS" career pages are not truly bespoke — they still expose structured, dated job data you can read generically.

Tier	What it tries	Why it comes first
0	**Known ATS signature**	Expand the dictionary to dozens of platforms — still a single structured request.
1	**Embedded structured data**	JobPosting JSON-LD (common thanks to Google for Jobs), __NEXT_DATA__ / embedded JSON, RSS, sitemaps — clean, dated, no custom code.
2	**Internal JSON / XHR API**	Most career pages are SPAs that fetch JSON; read that endpoint instead of scraping rendered HTML.
3	**AI-generated extractor**	An agent writes and verifies a custom extractor, then caches it as a new connector — only for the few sites the tiers above miss.

Generate, then verify and self-heal. A custom extractor written once will silently break when a site is redesigned, so the loop matters more than the generation. At generation time the agent must prove the extractor works — at least one job, required fields present, dates that parse, a plausible count — before it is ever saved. On every run the output is health-checked: zero jobs where there were twenty, or fields suddenly empty, triggers a quarantine and an automatic rebuild rather than feeding bad data downstream. A stored structure hash and a "last verified" date make drift easy to detect.

> **Two choices keep this safe and cheap:** the agent fills a constrained extractor recipe — which container, which fields, how to parse the date — rather than emitting free-form code to run blindly; and the "build a custom extractor?" decision is made **once per company** and stored — never a recurring prompt that would stall an unattended run.

06 The matching engine

Matching runs in two stages, for both quality and cost.

First, a cheap prefilter narrows the field. Your CV and free-text preferences are turned into a compact representation; each new posting is compared against it with lightweight similarity and simple rules (title family, seniority, location). Obvious non-matches are dropped before any expensive model sees them.

Second, a language model scores what survives. For each shortlisted posting it produces a fit score, two or three sentences on why the role fits you, and any concerns — a seniority mismatch, say, or a missing must-have skill. Because only plausible roles reach this stage, the model does meaningful work on a few postings rather than grading hundreds of irrelevant ones.

> **Your own words count.** Alongside the CV you keep a short note — for example, "I am a Data Scientist but would also consider roles leaning toward BI or Analytics Engineering."

> The matcher treats this as part of your profile, so adjacent roles you would actually take are surfaced, and ones you would not are filtered out. Editing that sentence instantly reshapes what you see.

What one recommendation looks like

Field	Example
Role	Senior Data Scientist
Company	Personio
Location	Munich (Hybrid)
Posted	2026-05-28
Fit score	88 / 100
Why it fits	Strong match on Python, experimentation, and product analytics; aligns with your openness to BI.
Watch-outs	Asks for 5+ years; you have 3.
Status	New

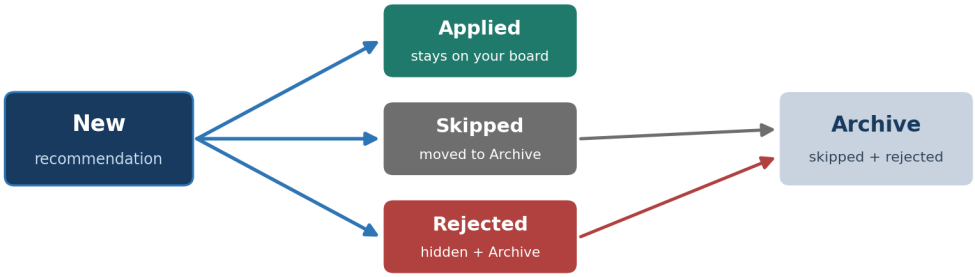
07 The application: your dashboard

The front end is deliberately simple — one page you actually want to open.

Setup, done once. You upload your CV (stored and reusable, editable whenever it changes) and write your free-text preferences. You maintain the company-and-location list. That is the entire configuration.

The board. Recommendations appear in a clean, sortable table — role, company, location, posting date, fit score, a one-line rationale, and a status. You act in place: mark a role Applied, Skipped, or Rejected, and jot notes in a dedicated column.

How status behaves. Applied roles stay on your board so you can track them. Rejected roles disappear from the main view immediately, keeping it focused. Skipped and Rejected roles are never lost — both live in an Archive you can browse or restore from.



Rejected vanishes from your main board; Skipped and Rejected stay browsable in the Archive.

The data model underneath

Four simple records run the whole app:

Record	Holds	Key fields
Profile	Your CV and preferences	`cv_text, preferences_note, updated_at`
Company	A target employer	`name, ats_type, ats_token, locations`
Posting	A job pulled from a feed	`job_id, title, location, posted_date, url, description`
Recommendation	A scored posting + your decision	`posting_id, fit_score, rationale, status, notes, first_seen`

Because `Posting.job_id` is stable and comes straight from the source feed, your decisions on each Recommendation persist correctly every time the pipeline runs again.

08 Triggers & automation

The pipeline is designed to run unattended.

Default: a daily schedule. A single scheduled run — say, early each morning — pulls new postings, matches them, and refreshes your board so new roles are waiting when you open it. No interaction required.

On demand: from your phone. When you want to trigger a run yourself — after updating your CV, or between calls — a phone trigger fits naturally. Tools such as Clawdbot / OpenClaw bridge a messaging app (WhatsApp, Telegram, and others) to commands on your machine, so a single message like “run jobscout” kicks off the pipeline and reports back when the shortlist is ready.

Built to run incrementally. Each run only fetches and scores what is new, deduplicated against everything already stored. That is what keeps every run fast and the cost negligible — the subject of the next section.

09 The first proof of concept

The goal of the POC is to prove the full loop end to end on a small scale — not to reach 500 companies on day one.

Scope

- 20–30 companies you genuinely care about.
- Three ATS connectors that between them cover most of that set: Greenhouse, Lever, and Personio.
- Your real CV and a one-paragraph preferences note.
- A minimal dashboard: a single table with status buttons and a notes field.

Build order

1. **Connectors.** Implement and test the three feed connectors; confirm each returns titles, locations, dates, and stable IDs.
2. **Storage + dedupe.** Store postings keyed by job ID; verify nothing is processed twice across runs.
3. **Matching.** Add the prefilter, then the LLM scoring pass; eyeball the rationales on real postings.
4. **Dashboard.** Render the recommendations table; wire up Applied / Skipped / Rejected and the Archive.
5. **Trigger.** Put the pipeline on a daily schedule; confirm new roles appear on their own.

> **Success criteria:** after a few days running unattended, the board shows genuinely relevant, correctly dated new roles from your target companies, your status decisions stick across runs, and you have stopped checking those career pages by hand.

10 What it costs

The economics are favourable because the design avoids the expensive paths.

The feeds are free. Pulling structured job data from ATS platforms costs nothing per request.

AI runs only on new postings. Thanks to deduplication, the model scores the handful of genuinely new roles each day — typically tens across a few hundred companies, not thousands every run.

Order of magnitude. Using a fast, inexpensive model (e.g. Claude Haiku 4.5 at $1 per million input tokens and $5 per million output, with batch and caching discounts available), scoring a few hundred new postings a day costs well under a euro a day — on the order of a few euros a month. The two-stage prefilter pushes this lower still.

Item	Cost
ATS feed pulls	Free
LLM matching (Claude Haiku 4.5)	$1 / $5 per million tokens (in / out); ~half with batch; cached input up to ~90% cheaper
Typical daily spend	Well under €1/day for a few hundred companies
Hosting	A small scheduled job or always-on script; minimal

Pricing per Anthropic's published rates, June 2026. The only way to make this expensive is to run a full browser agent against every company on every run — keeping that to the long-tail exception holds the cost near zero.

11 Risks & mitigations

Risk	Mitigation
Long-tail companies with no ATS feed	A fallback cascade — structured data (JSON-LD), then the page's own JSON API, then an AI-generated extractor — covers most; only true exceptions are skipped.
A generated extractor breaks after a site redesign	Output is health-checked each run; on an anomaly the extractor is quarantined and rebuilt automatically, escalated to you only if the rebuild fails.
A company switches or misconfigures its ATS	Connectors fail loudly and trigger re-classification; the one-time detection step is cheap to re-run.
Rate limits / terms of service on non-API sites	Prefer official feeds (well within normal use); throttle and cache the fallback; respect robots and ToS.
Unreliable dates from raw scrapes	Sidestepped by design — ATS feeds carry authoritative posting dates; the fallback flags low-confidence dates.
Duplicate or stale recommendations	Stable job IDs + dedupe; postings that vanish from a feed are marked closed.
Match-quality drift	Keep the rationale visible to sanity-check; tune the prefilter and prompt against your own Applied / Rejected history.
Maintenance creep	A small, well-bounded set of connectors is far cheaper to maintain than hundreds of scrapers.

12 Roadmap

Phase	Focus	What ships
1 · Prove the loop	POC	20–30 companies, 3 connectors, basic matching, a simple dashboard, a daily schedule.
2 · Scale & sharpen	Coverage + quality	Grow to the full list (hundreds); add remaining connectors + the fallback extractor; add the embedding prefilter; polish status, archive, and notes.
3 · Automate & refine	Hands-off + extras	Phone trigger; tune the matcher against your decisions; optional salary parsing, remote/location filters, and a daily digest.

> **Bottom line:** the idea is sound and the build is very achievable. The one decision that makes or breaks it — using structured ATS feeds instead of scraping — is also the one that makes it cheap, reliable, and low-maintenance. Start with the small POC, prove the loop, and grow the company list from there.

A Appendix A — ATS connector reference

A developer quick reference for the public feeds, verified against each platform's official documentation (June 2026). Replace {co} / {tenant} / {site} with the company's own identifiers, taken from its live career URL.

Platform	Method + endpoint	Returns
Greenhouse	`GET boards-api.greenhouse.io/v1/boards/{co}/jobs?content=true`	All jobs with full descriptions, departments, offices, updated_at.
Lever	`GET api.lever.co/v0/postings/{co}?mode=json`	Postings with categories (team, location, commitment), hosted + apply URLs, createdAt.
Personio	`GET {co}.jobs.personio.de/xml?language=en`	XML feed: title, office, department, employment type, description; multilingual.
Ashby	`GET api.ashbyhq.com/posting-api/job-board/{co}`	Whole public board as JSON; add ?includeCompensation=true for pay.

Platform	Method + endpoint	Returns
SmartRecruiters	`GET api.smartrecruiters.com/v1/companies/{co}/postings`	Published postings; filter by city, country, department, language; paginated.
Workday	`POST {tenant}.wdN.myworkdayjobs.com/wday/cxs/{tenant}/{site}/jobs`	JSON body {searchText, limit, offset, appliedFacets}; then GET .../job/{path} for detail.

B Appendix B — suggested tech stack

Nothing here is prescriptive — the architecture is deliberately simple and stack-agnostic. A pragmatic starting point:

- **Pipeline & connectors.** Python (requests + a little parsing), or Node.
- **Storage.** SQLite to start (a single file); Postgres if it grows.
- **Matching.** An embeddings model for the prefilter; a small LLM (e.g. Claude Haiku) for scoring.
- **Dashboard.** A lightweight web app — Streamlit, or a small Next.js / React front end over a simple API.
- **Scheduling.** A cron job or hosted scheduled task; add a Clawdbot / OpenClaw bridge for phone triggers.

The guiding principle: keep each piece small and replaceable. The connectors and the dedupe/state layer are the parts worth building carefully; everything else can stay minimal.